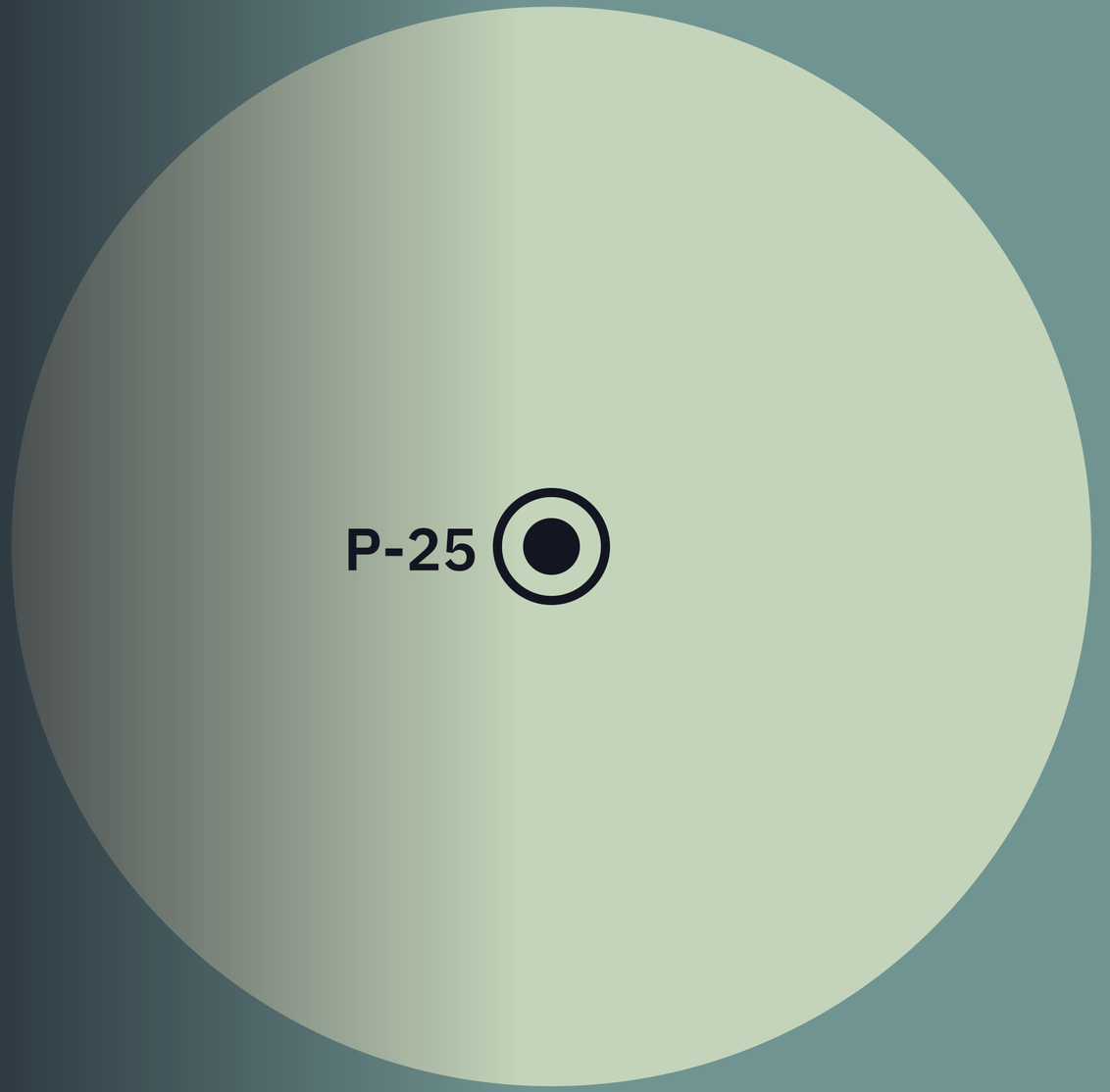
SignalMap

Hito 2: motor back-end y API REST

Mide la señal WiFi en puntos fijos de un edificio, marcados con tiras NFC, para comprobar si un repetidor mejoró la cobertura. Hoy mostramos el servidor.



Maximiliano Alvarado y Gabriel Díaz
Universidad Mayor · Arquitectura de Desarrollo (Web y Móvil)
Noviembre de 2026

Tres partes, en este orden

1

Demostración en vivo

Cuatro peticiones contra el servicio corriendo.

2

Tecnologías

Qué elegimos, frente a qué, y qué nos costó.

3

Arquitectura

Por qué un solo servicio, y cuándo lo revisaríamos.

Demostración en vivo: la vida del P-25

Un punto de medición nace, se corrige, se borra y deja de existir. **Cada paso responde con un código distinto.**

POST `/api/v1/points` — **201 Created**

Crea el punto y dice dónde quedó: `Location: /api/v1/points/25`

PATCH `/api/v1/points/25` — **200 OK**

Corrige solo `x` e `y`, sin reenviar todo el punto

DELETE `/api/v1/points/25` — **204 No Content**

Borra y no devuelve cuerpo

GET `/api/v1/points/25` — **404 Not Found**

Prueba de que el borrado fue real

391 tests, uno por cada respuesta del contrato, corren en cada Pull Request.

Seis preguntas antes de guardar un punto

Son como puertas en fila: cada una hace **una sola pregunta** y, si la respuesta es no, corta ahí y devuelve su código.

middlewares → controller → service → repository

routes/points.ts

```ts
pointsRouter.post(
  '/',
  authenticate,
  requireRole('admin', 'tecnico'),
  validate({ query: noQuery, body: createPointBody }),
  points.create,
);
```

1 **¿Quién eres?** `401 · sin sesión`

2 **¿Puedes crear puntos?** `403 · sin permiso`

3 ¿El cuerpo tiene forma?

Zod revisa el formato: x e y son números con hasta 2 decimales.

Si la respuesta es no:

`400 · mal formado`

```
{ "message": "x debe ser un número con hasta 2 decimales" }
```

Seis preguntas antes de guardar un punto

Son como puertas en fila: cada una hace **una sola pregunta** y, si la respuesta es no, corta ahí y devuelve su código.

middlewares → **controller** → service → repository

controllers/points.ts (sin tipos)

```ts
export async function create(req, res) {
  const point = await pointsService.create(req.user!, req.body);

res.status(201).location(`${req.baseUrl}/${point.id}`).json(point);
}
```

→ Dos líneas, ninguna decisión

Le pide al servicio que cree el punto y responde. Nada más.

Si todo salió bien:

`201 · creado`

`Location: /api/v1/points/25`

Seis preguntas antes de guardar un punto

Son como puertas en fila: cada una hace **una sola pregunta** y, si la respuesta es no, corta ahí y devuelve su código.

middlewares → controller → **service** → repository

services/points.ts (sin tipos)

```
function assertInside(floor, x, y) {
  const inside = (value, max) => value === undefined || (value >= 0
&& value <= max);
  if (!inside(x, floor.width_m) || !inside(y, floor.height_m)) {
    throw new AppError('domain_rule', `Las coordenadas quedan fuera
del piso (${floor.width_m} x ${floor.height_m} m)`);
  }
}
```

4 **¿Lo ve este usuario?** `404 · no lo ve`

5 ¿Cae dentro de 40 × 25 m?

Compara x e y con las medidas del piso. Con x = 45 no entra.

Si la respuesta es no:

`422 · rompe una regla`

```
{ "message": "Las coordenadas
quedan fuera del piso (40 x 25 m)"
}
```

Seis preguntas antes de guardar un punto

Son como puertas en fila: cada una hace **una sola pregunta** y, si la respuesta es no, corta ahí y devuelve su código.

middlewares → controller → service → **repository**

repositories/points.ts (sin tipos)

```
const { rows } = await query(
  `INSERT INTO points (floor_id, code, x, y, nfc_uid) VALUES ($1, $2, $3, $4, $5)
   RETURNING id, floor_id, code, x, y, nfc_uid`,
  [point.floor_id, point.code, point.x, point.y, point.nfc_uid],
);
```

6 ¿Ya existe P-25 o esa tira?

La base lo impide sola con UNIQUE. Los valores viajan como parámetros ($1, $2…), nunca pegados al SQL.

Si la respuesta es sí:

409 · duplicado

```
{ "message": "La tira NFC ya está asociada a otro punto" }
```

Seis preguntas antes de guardar un punto

Son como puertas en fila: cada una hace **una sola pregunta** y, si la respuesta es no, corta ahí y devuelve su código.

`middlewares` → `controller` → `service` → `repository`

```
terminal · desde backend/src

$ grep -l "from 'express'" services/*.ts
(ninguno)
$ grep -l "res\.\|\.status(" services/*.ts
(ninguno)
$ grep -l "repositories/" controllers/*.ts schemas/*.ts
(ninguno)
$ grep -rl "SELECT \|INSERT \|UPDATE \|DELETE FROM" . | grep -v
repositories/
(ninguno)
```

¿Capas o solo carpetas?

Cuatro búsquedas en el código: ningún service conoce Express, ningún controller toca la base y no hay SQL fuera de repositories/.

Resultado:

`4 de 4 en «ninguno»`

Express, frente a cuatro alternativas

Los mismos criterios para todo: **un equipo de 2 personas y un semestre**, el lenguaje de la web (TypeScript), capas que se puedan defender, que **los dos** podamos explicar cada parte, y una carga baja (~24 puntos por piso).

Express 5

Elegido

Rutas y middlewares, sin estructura impuesta. Muy documentado, y se integra directo con Zod y Supertest.

Costo: Express no impone capas. La separación es una disciplina nuestra, y la comprobamos con búsquedas en el código.

Fastify

La velocidad no se nota con esta carga, y suma un sistema de plugins por aprender.

NestJS

Para 2 personas y 2 recursos, la ceremonia cuesta más de lo que ordena.

Hono

Su fuerte son las funciones en el borde de la red, que no usamos.

Spring Boot

Otro lenguaje: no comparte tipos con la web en TypeScript.

Cada elección, con su costo

TypeScript sin compilar

en vez de compilar a `dist/`

Node 26 corre los `.ts` directo y `tsc` solo revisa tipos: lo que corre es lo que está en el repositorio.

Costo: nada de `enum` ni `namespace`, y el proyecto depende de Node 26.

pg con SQL a mano

en vez de Prisma, Knex o Kysely

Las consultas quedan a la vista en `repositories/`, que es la evidencia de las capas. Prisma pediría su propio esquema, duplicando `db/schema.sql`.

Costo: mapear filas a objetos y declarar los tipos a mano.

JWT propio con node:crypto

en vez de `jsonwebtoken`

Cero dependencias en la pieza más sensible del servicio.

Costo: verificar el token es cosa nuestra, así que hay 28 tests de ataques (`alg: none`, firma alterada, vencido, deformes). Todos responden `401`.

Vitest

en vez de Jest

Trae TypeScript y ESM sin configurar. El ESM de Jest es experimental y pide una bandera de Node.

Costo: también transforma el TypeScript por su cuenta; lo cubre `npm run typecheck` en el CI.

PostgreSQL, frente a MongoDB (Hito 1): los datos son una cadena y la base cuida su integridad sola, con claves foráneas, `UNIQUE` y `CHECK`. **Costo:** esquema rígido, con migraciones.

Un solo servicio, por ahora

Un **monolito** es un único programa que hace todo. Lo dividiríamos solo si algún día hiciera falta.

2

personas y un semestre para construirlo y operarlo

~24

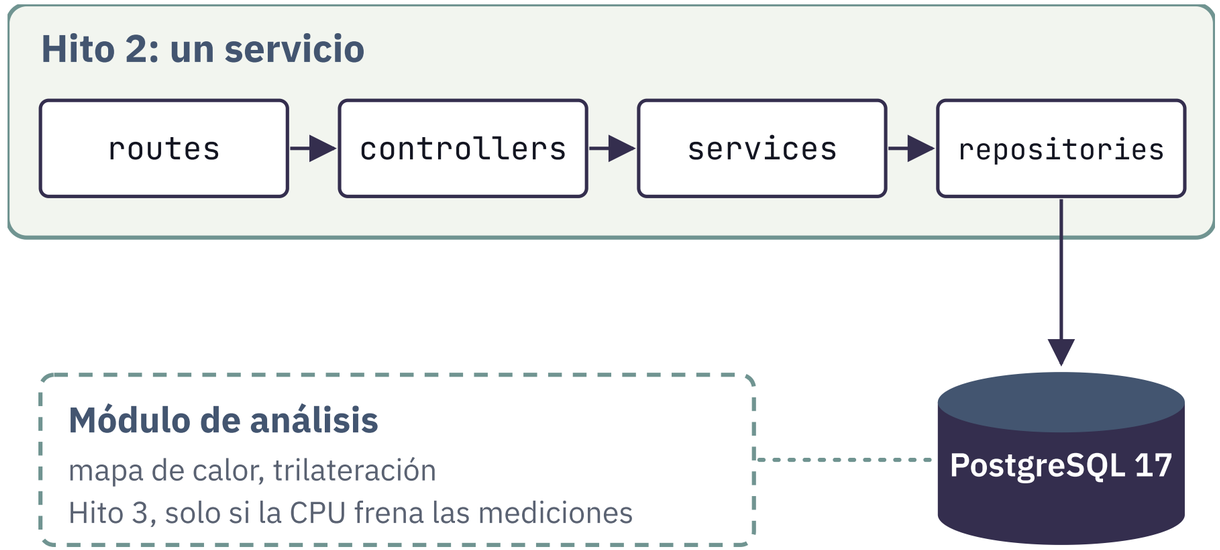
puntos por piso, y el técnico mide de a uno

4

escaneos WiFi cada 2 minutos permite Android

1

proceso y una base, y un solo `/health` que vigilar



> **Costo asumido:** todo escala junto, y un error grave puede tumbar el servicio entero.

> **Se revisa si** el análisis del Hito 3 consume tanta CPU que frena el registro de mediciones. Eso se separa primero, porque solo lee datos.

Fowler (2015): los microservicios exitosos casi siempre empezaron como un monolito.

¿Preguntas?

13

endpoints, con CRUD completo de edificios y puntos

7

tablas, que un script crea desde cero

391

tests, uno por cada respuesta del contrato

86 / 86

fallas introducidas a propósito, todas detectadas

